STUDENT ACADEMIC MANAGEMENT PLATFORM

Project Documentation & Technical Report

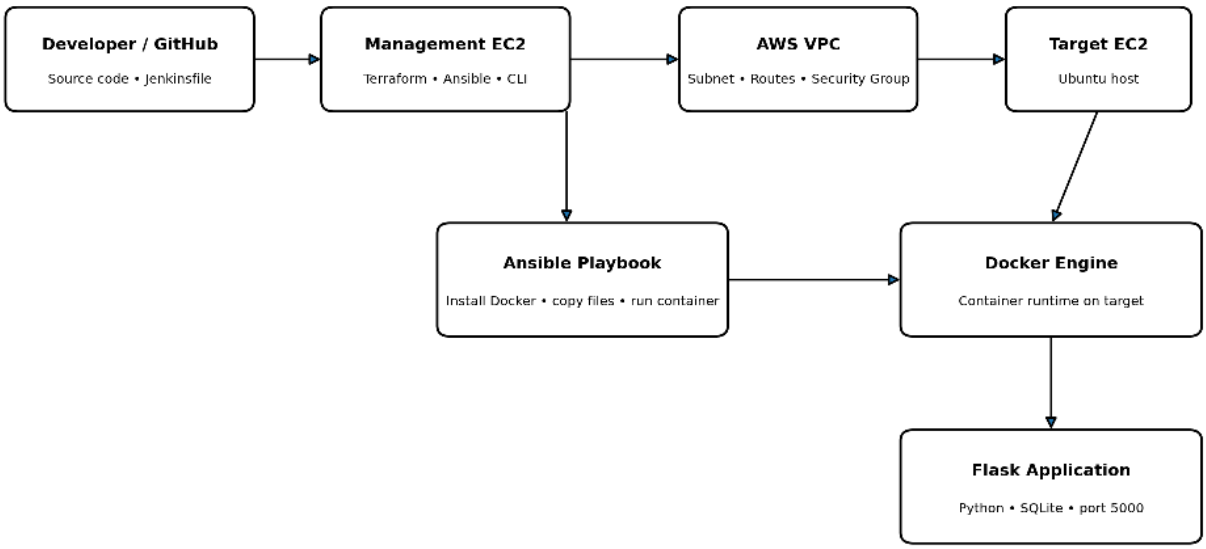


Figure 1. High-level project architecture (logical view).

Technology focus: AWS • Terraform • Ansible • Docker • Flask • SQLite • Jenkins • Kubernetes

Repository: https://github.com/anushkarajdev2410-netizen/student-academic-platform

Document Purpose and Scope

This document records the purpose, design, architecture, application structure, deployment model, testing approach, security considerations, limitations, and future improvements for the Student Academic Management Platform. It is intended to complement—not duplicate—the separate step-by-step implementation PDF. Commands and detailed installation procedures are therefore not repeated here.

Evidence note: this report distinguishes confirmed project details from items that still require a final live check. The repository and deployment configuration discussed during implementation include a Flask application, SQLite storage, a Dockerfile, Terraform configuration, an Ansible inventory/playbook, a Jenkinsfile, tests, and Kubernetes manifests. Final runtime status, test pass counts, API responses, Jenkins build status, and Kubernetes replica health should be filled in from actual command output before submission.

Project at a Glance

Item	Description
Project title	Student Academic Management Platform
Project type	Cloud-hosted web/API application with infrastructure automation
Primary language	Python
Web framework	Flask
Database	SQLite (file-based database; persistent directory intended for data storage)
Containerization	Docker
Infrastructure automation	Terraform
Configuration/deployment automation	Ansible
CI/CD configuration	Jenkinsfile in repository; pipeline status must be verified
Orchestration configuration	Kubernetes manifests in repository; live cluster status must be verified
Source control	Git and GitHub
Repository	anushkarajdev2410-netizen/student-academic-platform

1. Abstract

The Student Academic Management Platform is a web-based application concept for organizing core academic records such as students, faculty, courses, enrollments, marks, and GPA-related information. The project combines application development with DevOps practices to demonstrate how software can be version-controlled, packaged, deployed, and managed on cloud infrastructure. The application uses Python and Flask for its web/API layer and SQLite for lightweight data persistence. Docker provides a repeatable runtime package, while Terraform describes AWS infrastructure declaratively and Ansible automates target-server configuration and deployment tasks. A Jenkins pipeline and Kubernetes manifests are included in the project scope for CI/CD and orchestration. This documentation summarizes the system design, components, security considerations, verification approach, limitations, and potential future work. Live deployment and pipeline claims should be supported by captured test and runtime evidence.

2. Problem Statement

Academic information can become difficult to maintain when student details, course registrations, faculty information, marks, and GPA calculations are spread across separate files or handled manually. Manual record handling can lead to duplicate data, inconsistent updates, slow retrieval, and avoidable administrative effort. In parallel, deploying a software application manually on a cloud server can introduce configuration differences and make the environment harder to reproduce. This project addresses these concerns by pairing an academic-record application with a repeatable DevOps deployment approach.

3. Objectives

Provide a central application structure for managing student and academic records.

Expose application functionality through a Flask-based web/API layer.

Use SQLite as a lightweight persistence mechanism for the prototype.

Package the application and its Python dependencies in a Docker image.

Define AWS infrastructure using Terraform configuration files.

Automate server setup and application deployment with Ansible.

Use GitHub for source control and maintain a Jenkinsfile for pipeline automation.

Maintain Kubernetes deployment/service manifests as part of the orchestration scope.

Validate the project with automated tests, service checks, and deployment evidence.

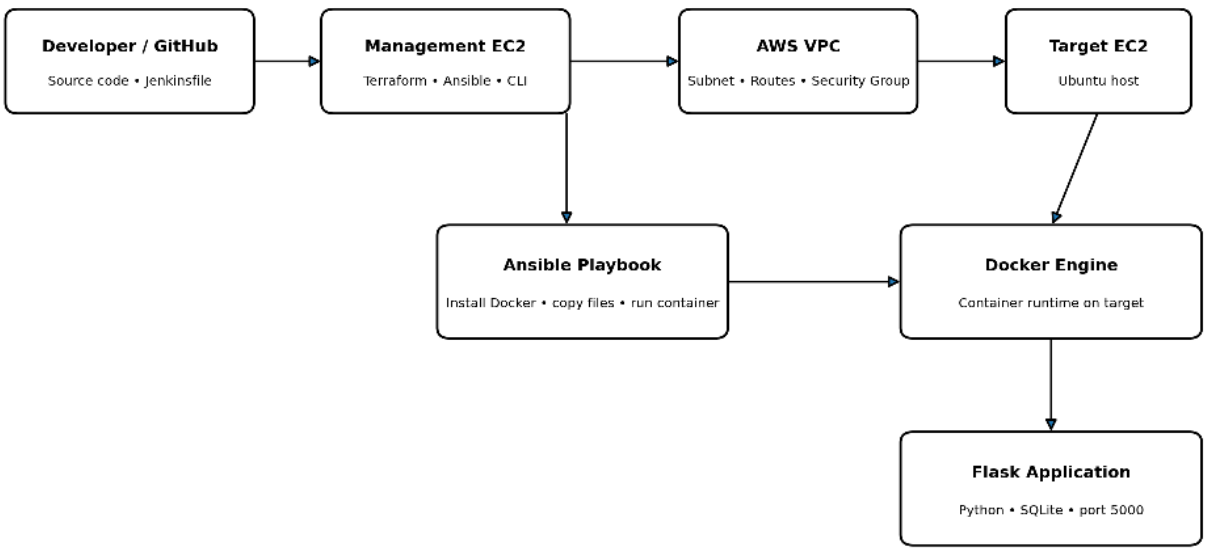
4. Functional Scope

Area	Intended capability	Verification note

Students	Create, retrieve, and manage student records	Confirm supported operations in app routes/tests
Faculty	Maintain faculty-related records	Confirm supported operations in app routes/tests
Courses	Maintain course information	Confirm supported operations in app routes/tests
Enrollments	Associate students with courses	Confirm validation and relationship handling
Marks	Record or retrieve academic marks	Confirm actual route and request schema
GPA	Support GPA-related academic calculations or display	Confirm calculation rules and edge cases
Health/API checks	Provide endpoints for service verification	Use actual routes in code; do not assume endpoint names

The feature list above reflects the project scope discussed during implementation. Exact HTTP methods, URL paths, field names, validation rules, and response formats must be taken from the current application source code and tests.

5. System Architecture



Application access in the current deployment plan: host port 8080 → container port 5000

Figure 2. Logical architecture for the current project setup. The host-port mapping shown is the planned Ansible deployment configuration.

The developer maintains the code in GitHub. A management/control EC2 environment contains the cloud and automation toolchain. Terraform defines AWS resources, while Ansible connects to the target EC2 host over SSH and configures it. The target host runs Docker, which launches the Flask application. The Dockerfile specifies Python 3.12 slim, installs requirements, copies the app and run.py, sets DATABASE=/app/database/student_platform.db, exposes container port 5000, and starts the application using python run.py. The Ansible playbook discussed for this project maps host port 8080 to container port 5000; public access works only if the target security group allows the chosen source and port.

5.1 Application request path

1. A client sends an HTTP request to the target EC2 host on the configured public port.
2. The Docker port mapping forwards traffic from host port 8080 to container port 5000.
3. Flask receives the request and routes it to the relevant application handler.
4. The application reads or updates SQLite data using the configured database path.
5. Flask returns an HTTP response to the client.

6. Technology Stack

Technology	Role in the project	Notes
Python 3.12	Application runtime	Specified by the Docker base image
Flask 3.1.1	Web/API framework	Version listed in requirements.txt
SQLite	Prototype data store	Database file path is configured through DATABASE environment variable
Docker	Container packaging/runtime	Dockerfile exposes port 5000
AWS EC2	Cloud compute	Management/control and target host roles were used during implementation
AWS VPC / subnet / routes / security group	Network isolation and access control	Resources are managed through Terraform
Terraform	Infrastructure as Code	Declares AWS resources and records them in Terraform state
Ansible	Configuration and deployment automation	Uses SSH to configure the target host
Jenkins	CI/CD automation	Jenkinsfile exists; final

		successful build must be evidenced
Kubernetes	Container orchestration	Manifests exist; live cluster state must be verified
Git / GitHub	Version control and collaboration	Repository is the source of truth for tracked files
pytest / pytest-cov / Ruff	Testing and code-quality tooling	Listed in requirements.txt

7. Infrastructure Design and Automation

7.1 Terraform responsibilities

Terraform describes the AWS infrastructure as code. The project's infrastructure work has included a VPC, subnet, internet gateway, route table, security group, and EC2-related resources. Terraform state associates declared resources with the corresponding AWS objects. The state file is operationally sensitive and must not be committed to a public repository. Review a plan before applying changes, and avoid terraform destroy unless intentionally decommissioning the environment.

7.2 Ansible responsibilities

Ansible uses an inventory to identify the target EC2 host and connect over SSH. The deployment playbook is intended to install Docker and curl, enable the Docker service, copy the application files, build the image, replace the named container, start it with a restart policy and persistent database-directory mount, and wait for the application port. A successful Ansible ping proves connectivity only; it does not by itself prove that the application or all API routes work.

7.3 Network and port model

Traffic	Port	Purpose / consideration
SSH administrator access	22	Restrict source to the administrator's IP or a controlled network.
Application access on EC2 host	8080	Used by the proposed Ansible host-to-container mapping; security group must permit intended clients.
Flask inside the container	5000	Port declared by the Dockerfile and used by run.py.

The trainer manual's reference architecture uses a different network CIDR and mentions Nginx, S3 backups, and remote Terraform state. These are reference-design elements, not proof that every one of them was implemented in this student deployment. Describe them as future enhancements unless the AWS console and repository confirm they are present.

8. Container and Data Design

The Docker image uses python:3.12-slim as its base, installs the packages in requirements.txt, copies app/ and run.py, creates /app/database, sets the DATABASE environment variable, exposes port 5000, and launches the application. The Ansible deployment configuration mounts the target host's database directory into /app/database in the container. This is intended to preserve SQLite data across container replacement on that host. It is not a substitute for off-host backup, disaster recovery, multi-writer database support, or a managed database service.

9. CI/CD and Orchestration

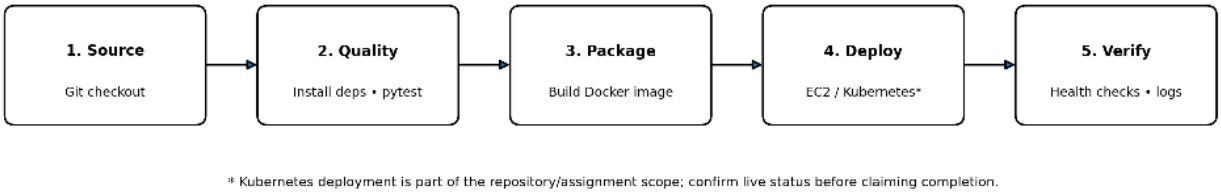


Figure 3. Conceptual CI/CD workflow; this diagram describes the intended stages, not a verified Jenkins build.

The repository contains a Jenkinsfile and Kubernetes manifests. A typical pipeline for this project checks out source, installs dependencies, runs automated tests, builds a Docker image, and deploys or prepares the application for runtime validation. Kubernetes manifests can define a Deployment and Service, including replica count and network access. The actual stage names, image tags, cluster context, service type, and replica count must be confirmed from the current Jenkinsfile and manifests before they are reported as completed.

9.1 Deployment verification checklist

Jenkins: latest build is successful and all intended stages are green.

Docker: the application container is running and the port mapping is correct.

Application: curl returns an expected response from a real route.

Kubernetes (if deployed): Deployment is available, expected replicas are ready, and the Service is present.

Recovery: confirm a pod can be recreated after deletion only if the Kubernetes deployment is actually running.

Evidence: capture console output or screenshots with timestamps and resource names.

10. Testing and Quality Assurance

The project requirements include pytest, pytest-cov, and Ruff. Unit and integration tests should cover valid and invalid inputs, CRUD behavior, data persistence, status codes, and academic calculation logic where implemented. Coverage output can help identify untested paths, while linting can flag style and common code-quality issues. Dependency installation alone is not a test result; the final report should include the real pytest summary.

Test layer	Suggested evidence	Result to record
Unit tests	pytest -v output	Passed / failed count: _____
Coverage	pytest --cov=app --cov-report=term-missing	Coverage: _____
Code quality	ruff check .	Issues / result: _____
Container health	docker ps and container logs	Running state: _____
HTTP/API	curl against actual endpoint(s)	Endpoint and HTTP status: _____
CI/CD	Jenkins console output	Build number and status: _____
Kubernetes	kubectl get deployments,pods,services	Ready replicas / Service: _____

10.1 Assessment rubric

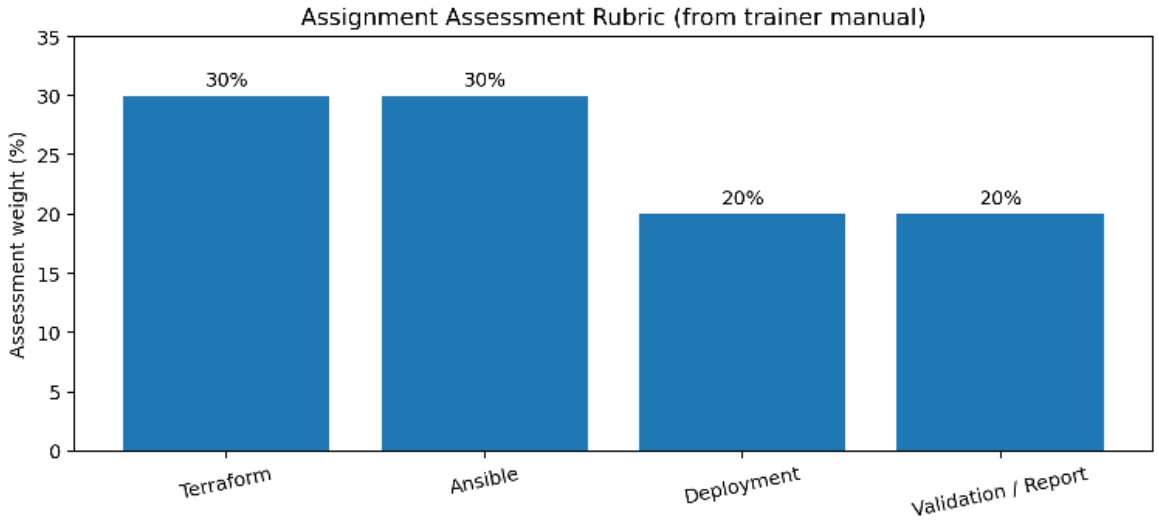


Figure 4. Assessment weights reproduced from the trainer implementation manual: Terraform 30%, Ansible 30%, deployment 20%, validation/ report 20%.

These percentages are the rubric shown in the supplied trainer manual. They are not a claim about the user's earned score.

11. Security Considerations

Never commit private SSH keys, AWS access keys, .env secrets, or Terraform state to GitHub.

Use IAM roles or narrowly scoped credentials rather than the AWS root user for routine automation.

Restrict SSH inbound access to a known administrator IP range whenever possible.

Expose only required application ports and avoid opening management services to the internet.

Keep the operating system, Docker engine, Python dependencies, and base image updated.

Use a private, versioned, encrypted backup destination for important database backups when implementing backup capability.

Avoid storing real student personal information in a demonstration environment; use synthetic records.

Use HTTPS and appropriate authentication/authorization before exposing the platform to real users.

12. Limitations

SQLite is suitable for a small prototype but has limitations for concurrent writes and horizontally scaled application instances.

A single EC2 host is a single point of failure unless redundancy and recovery mechanisms are added.

A host-mounted database directory protects against container replacement on that host but does not provide disaster recovery.

Public HTTP access without TLS and strong authentication is not appropriate for real academic records.

The exact endpoint contract, field validation, GPA calculation rules, and authorization controls require verification from source code and tests.

Jenkins and Kubernetes configuration files do not prove a live successful pipeline or cluster deployment; runtime evidence is required.

Operational costs may continue for EBS volumes, public IPv4 addresses, snapshots, NAT gateways, or other resources even when EC2 instances are stopped.

13. Future Enhancements

Move from SQLite to Amazon RDS PostgreSQL when concurrent access and managed backups are required.

Add authentication, role-based access control, audit logging, input validation, and secure session management.

Serve traffic through Nginx or an AWS load balancer and configure HTTPS certificates.

Add automated database backups, retention policies, and tested restore procedures.

Use a remote Terraform backend with encryption, access controls, versioning, and locking where appropriate.

Add vulnerability scanning, dependency pinning, image scanning, and automated quality gates to Jenkins.

Use Kubernetes health probes, resource requests/limits, secrets management, and rolling update policies if a

cluster is part of the final deployment.

Add structured logging, monitoring, alerts, and documented incident/recovery procedures.

14. Conclusion

The Student Academic Management Platform combines a Flask-based academic application with cloud and DevOps automation. Its documented design uses Terraform for infrastructure definition, Ansible for target-server configuration, Docker for repeatable packaging, and repository-based Jenkins/Kubernetes configuration for the broader delivery workflow. The project demonstrates how application code and infrastructure can be managed together. Before final submission, the report should be completed with actual test results, successful deployment evidence, verified API responses, and Jenkins/Kubernetes status. Only components that are demonstrably deployed should be described as live.

Appendix A. Project File Inventory

Path / file	Purpose
app/	Flask application package and feature modules
database/	Database-related project directory
tests/	Automated test files
terraform/	Terraform configuration and state-related local files
ansible/inventory.ini	Ansible target inventory
ansible/site.yml	Ansible deployment playbook discussed for this setup
Dockerfile	Container image definition
requirements.txt	Python dependencies, including Flask and test/ quality tools
run.py	Application entry point; creates Flask app and runs on 0.0.0.0:5000
Jenkinsfile	Jenkins pipeline configuration
k8s/	Kubernetes deployment/service manifests
README.md	Repository introduction and usage documentation

Appendix B. Evidence to Attach Before Submission

GitHub repository page showing the project files and latest commit.

Terraform state list and resource summary (hide sensitive values).

Ansible playbook successful recap and target ping result.

Docker container status and application logs.

Successful pytest output and coverage summary.

HTTP response from a real application endpoint.

Jenkins successful build with stage results.

Kubernetes deployment, pod, and service output if applicable.

AWS console screenshot showing the intended EC2 and security group configuration.

Source Basis

This document was prepared from the project details shared during the implementation conversation, the repository/file structure discussed there, the observed Dockerfile and requirements.txt contents, and the supplied Careertiq Unit 4 trainer implementation manual and hands-on lab guide. Where live state or test output was not available, the report labels it as requiring verification instead of inventing a result.